Self-Paced Module: Layouts and Panels

Java Programming · Grade 12 TVL · Cielito Zamora Senior High School
Week 18 · November 16–20, 2026 · Continues Week 17 Swing Basics

Name: _____ Section: _____ Date: _____

Day / time	Topic	Work and strip
Mon Nov 16 / 2 h	FlowLayout; JPanel groups	Page 1 Step 1; Monday strip, page 7 bottom.
Tue Nov 17 / 2 h	BorderLayout regions	Page 2 Step 2; Tuesday strip, page 7 middle.
Wed Nov 18 / 2 h	GridLayout; equal cells	Page 3 Step 3; Wednesday strip, page 7 top.
Thu Nov 19 / 3 h	Nested panels; own-theme start	Page 4 Step 4; help on page 5; task plan page 6; Thursday strip, page 8 bottom.
Fri Nov 20 / 3 h	Own-theme task; presentations	Page 5 Step 5; page 6 task; Friday strip, page 8 top.

How to use. Work in order, type and run the examples, predict changes, then record actual results. LIPLET means quiz strip; detach only today's strip. Pages 7–8 are single-sided.

Offline: JDK only; one named class per file, all in one folder. Compile `javac *.java`; run `java Main` for Vehicle. Step 1 uses `java LayoutDemo`; Step 5 uses `java ThemeLayoutCheck`. Error Guide: page 5. Close the old window before rebuilding.

Where we left off and what changes

Files in your folder	Keep / add / replace
Vehicle, Jeepney, Tricycle, Bus, OverloadException, InvalidDistanceException, VehicleNotFoundException (.java)	Retain all 7 model/exception files unchanged.
DispatchThread, BoardingTask, ReadyReport, ConvoyLeg, LoadingBay (.java)	Retain all 5 Week 16 helpers unchanged.
FareService.java, FareCheck.java, ReadyTask.java	Retain unchanged from Week 17 Step 5.
Main.java; TerminalPanel.java	Back up Main as Main_week17.txt before Step 2. Replace Main at Step 2; replace TerminalPanel at Steps 3–4.
LayoutDemo.java; ThemeLayoutCheck.java	Add at Steps 1 and 5. Keep Main_week16.txt; .txt backups do not compile. ThemeCheck was optional, not the final app.

Catch-up: use the complete Week 17 Step 5 starting files from the teacher's CodeShare copy. Keep the four sample vehicles and their constructor arguments; do not restart the model.

MONDAY | Make Room: FlowLayout and Panels

Definition. A layout manager is an object that decides the size and position of controls inside a container, an object that holds controls. A JPanel is a container for related controls. FlowLayout places controls in order at their preferred size, the size each control requests, and starts another row when needed [NTU] [HWS].

> **ANALOGY** Jeepneys join a row at the terminal. When the row is full, the next vehicle joins another row. A longer name still needs room; the manager does not make the letters smaller.

STEP 1 Try a row that wraps

Add LayoutDemo.java. Retain every Vehicle file. Compile javac *.java; run java LayoutDemo. This separate arrangement demo has no button actions. Run java Main to compare the Week 17 application.

LayoutDemo.java — first half

```
import java.awt.*;
import javax.swing.*;
public class LayoutDemo {
  public static void main(String[] args) {
    if (GraphicsEnvironment.isHeadless()) {
      System.out.println(
        "No display: GUI not opened.");
      return;
    }
    SwingUtilities.invokeLater(new Runnable() {
      public void run() {
        JPanel row = new JPanel(
          new FlowLayout(FlowLayout.LEFT, 8, 8));
        row.add(new JLabel("Terminal:"));
        row.add(new JButton("Jeepney"));
```

LayoutDemo.java — continued

```
        row.add(new JButton("Tricycle"));
        row.add(new JButton("Bus"));
        JFrame f = new JFrame(
          "Flow layout practice");
        f.setDefaultCloseOperation(
          JFrame.DISPOSE_ON_CLOSE);
        f.setContentPane(row);
        f.setSize(420, 180);
        f.setLocationRelativeTo(null);
        f.setVisible(true);
      }
    });
  }
}
```

Join both halves into one file. Observed without a display: `No display: GUI not opened.` On a school display, resize to see rows wrap. Appearance is an expected check, not an observed result here.

> **Check yourself.** Try FlowLayout.CENTER; predict and verify alignment, then restore LEFT. Narrow the window, then make it taller: wrapped rows need height. The demo buttons have no listeners; Vehicle's actions remain unchanged.

Sources [1 NTU] [2 HWS] [3 PRIN] [4 CMU] [5 NUS] (page 5). **Key validated facts:** FlowLayout can wrap controls and uses their requested size [HWS]; a JPanel groups controls and can change its manager [NUS].

> **LIPLET · MONDAY** After Step 1 on page 1, answer the MONDAY strip (bottom of page 7) in pen. Detach only today's strip and submit it.

TUESDAY | Give Each Group a Place: BorderLayout

Definition. BorderLayout divides a container into NORTH (top), SOUTH (bottom), EAST (right), WEST (left), and CENTER. Put one component or panel in each used region. The center receives the space left after the edges; unused regions take no space [HWS] [CMU].

ANALOGY Think of a school office: a heading above, service work in the middle, and a status counter below. Each counter can itself hold a group of controls.

Region	Size behavior	Vehicle use
NORTH / SOUTH	Full available width; requested height	Heading / readiness group
WEST / EAST	Requested width; remaining middle height	Unused this week
CENTER	All space left after edges and gaps	TerminalPanel

STEP 2 Give the terminal clear regions

First copy Main.java to Main_week17.txt. Replace Main.java below. Retain TerminalPanel.java and ReadyTask.java unchanged. Compile javac *.java; run java Main.

Main.java

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
public class Main {
  static JPanel createContent() {
    JPanel root = new JPanel(new BorderLayout(8, 8));
    root.setBorder(BorderFactory.createEmptyBorder(8, 8, 8, 8));
    TerminalPanel panel = new TerminalPanel();
    JLabel status = new JLabel("Not checked");
    JButton check = new JButton("Check ready");
    check.addActionListener(new ActionListener() {
      public void actionPerformed(ActionEvent e) {
        check.setEnabled(false);
        status.setText("Checking...");
        new Thread(new ReadyTask(status, check),
          "Readiness").start();
      }
    });
    JPanel ready = new JPanel(new FlowLayout(FlowLayout.LEFT, 8, 8));
    ready.add(check); ready.add(status);
    root.add(new JLabel("Vehicle Terminal"), BorderLayout.NORTH);
    root.add(panel, BorderLayout.CENTER);
    root.add(ready, BorderLayout.SOUTH);
    return root;
  }
  public static void main(String[] args) {
    if (GraphicsEnvironment.isHeadless()) {
      System.out.println("No display: GUI not opened.");
      return;
    }
    SwingUtilities.invokeLater(new Runnable() {
      public void run() {
        JFrame frame = new JFrame("Vehicle Terminal");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setContentPane(createContent());
        frame.setSize(720, 320);
        frame.setMinimumSize(new Dimension(520, 300));
        frame.setLocationRelativeTo(null);
        frame.setVisible(true);
      }
    });
  }
}
```

Read the structure. `createContent()` returns the root panel. It runs inside invokeLater on the event dispatch thread (EDT), the thread that handles interface work. createEmptyBorder adds an outer margin.

Preserve behavior. The readiness listener and ReadyTask stay the same. The new ready panel holds the button and status once, instead of appending them to TerminalPanel. Fare logic is untouched.

Observed without a display: `No display: GUI not opened.`

Check yourself. Name root's three children. Why must two controls share a group before going into CENTER? Test Quote fare during Check ready.

School-display check. Check heading, fare panel and readiness order. Try the 520 × 300 minimum and larger sizes. setMinimumSize limits shrinking; still check clipping. Test Tab, Shift+Tab and Space.

Sources [1 NTU] [2 HWS] [3 PRIN] [4 CMU] [5 NUS] (page 5). **Key validated facts:** BorderLayout gives remaining room to the center [CMU]; a child panel can contain several controls within one region [NUS].

LIPLET · TUESDAY After Step 2 on page 2, answer the TUESDAY strip (middle of page 7) in pen. Detach only today's strip and submit it.

WEDNESDAY | Line Up the Form: GridLayout

Definition. GridLayout arranges controls in equal cells, filling each row from left to right before starting the next row. Each cell holds one component. The order of add calls therefore determines the label-field pairs [NTU] [HWS].

> **ANALOGY** A class seating chart has equal spaces. Place a label and its field next to each other, like partners assigned to a row. A large result message is a poor fit for a tiny equal cell.

Constructor idea	What it does	Limit
3 rows, 2 columns, gaps 8 / 8	Six controls fill three rows	Buttons and result also stretch.
0 rows, 2 columns	Rows grow to keep two columns	Useful for a growing two-column form.
Rows greater than zero	Rows stay fixed; columns are computed	Adding a seventh control to 3 rows creates 3 columns.
0 rows, 0 columns	Invalid	At least one dimension must be nonzero.

STEP 3 Try equal cells

Replace TerminalPanel.java only. Keep Tuesday Main.java and all model, service, test and worker files. Compile javac *.java; run java Main.

TerminalPanel.java

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
public class TerminalPanel extends JPanel
    implements ActionListener {
  private JTextField plate =
    new JTextField("NXA 123", 8);
  private JTextField km = new JTextField("5", 5);
  private JLabel result =
    new JLabel("Enter plate and km");
  public TerminalPanel() {
    setLayout(new GridLayout(3, 2, 8, 8));
    JLabel p = new JLabel("Plate:");
    JLabel d = new JLabel("Distance (km):");
    p.setLabelFor(plate);
    d.setLabelFor(km);
    JButton quote = new JButton("Quote fare");
    quote.addActionListener(this);
    add(p); add(plate); add(d); add(km);
    add(quote); add(result);
  }
  @Override public void actionPerformed(
      ActionEvent e) {
    try {
      result.setText(FareService.quote(
        plate.getText(), km.getText()));
    } catch (NumberFormatException ex) {
      result.setText("Enter a number for km.");
    } catch (VehicleNotFoundException ex) {
      result.setText(ex.getMessage());
    } catch (InvalidDistanceException ex) {
      result.setText(ex.getMessage());
    }
  }
}
```

Observed console result

```
No display: GUI not opened.
```

Keep the rules. Only the layout import and constructor arrangement changed. The fields, `setLabelFor` links, listener registration, `FareService.quote` call and all three catches remain. InvalidDistanceException extends RuntimeException directly; keep its separate catch. Fares are classroom practice values, not LTFRB rates.

Input	Expected feedback to verify on the display
NXA 123 / 5	NXA 123: P15.00
NXA 123 / abc; then 5	Enter a number for km.; then NXA 123: P15.00
NXA 123 / 0; ZZZ / 5	Use a finite km above 0.; Unknown plate: ZZZ

> **Check yourself.** Predict the 3 × 2 arrangement: Plate / field; Distance / field; Quote fare / result. Notice the button and result are stretched. The temporary grid is useful for comparison; Thursday will separate unlike controls.

Sources [1 NTU] [2 HWS] [3 PRIN] [4 CMU] [5 NUS] (page 5). **Key validated facts:** GridLayout gives controls equal cells and follows add order [HWS] [NTU]; use a separate group when those equal sizes are unsuitable [NUS].

> **LIPLET · WEDNESDAY** After Step 3 on page 3, answer the WEDNESDAY strip (top of page 7) in pen. Detach only today's strip and submit it.

THURSDAY | Build a Clear Interface: Nested Panels

Definition. Nested panels are panels placed inside other panels. Each panel uses its own layout manager for its direct children. A parent treats a whole child panel as one component, so a form, an action row, and feedback can follow different arrangements [PRIN] [NUS].

ANALOGY A terminal contains counters; each counter contains its own forms and buttons. Changing the form arrangement does not change the fare rules or the loading-bay worker.

Parent → child group	Manager of direct children	Why it helps
root → heading / TerminalPanel / ready	BorderLayout	Different jobs occupy stable regions.
TerminalPanel → form / result / actions	BorderLayout	The result has room; button need not stretch.
form → two label-field pairs	GridLayout(2, 2, 8, 8)	Equal aligned cells for similar inputs.
actions → Quote fare; ready → check + status	FlowLayout	Small related controls sit in a row.

STEP 4 Nest panels for a useful form

Replace TerminalPanel.java only. Keep Tuesday Main.java and every other file. Compile javac *.java; run java Main. Run java FareCheck again to check the retained model.

TerminalPanel.java

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
public class TerminalPanel extends JPanel
    implements ActionListener {
  private JTextField plate =
    new JTextField("NXA 123", 8);
  private JTextField km = new JTextField("5", 5);
  private JLabel result =
    new JLabel("Enter plate and km");
  public TerminalPanel() {
    setLayout(new BorderLayout(8, 8));
    JLabel p = new JLabel("Plate:");
    JLabel d = new JLabel("Distance (km):");
    p.setLabelFor(plate);
    d.setLabelFor(km);
    JButton quote = new JButton("Quote fare");
    quote.addActionListener(this);
    JPanel form = new JPanel(new GridLayout(2, 2, 8, 8));
    form.add(p); form.add(plate);
    form.add(d); form.add(km);
    JPanel actions = new JPanel(new FlowLayout(FlowLayout.LEFT));
    actions.add(quote);
    add(form, BorderLayout.NORTH);
    add(result, BorderLayout.CENTER);
    add(actions, BorderLayout.SOUTH);
  }
  @Override public void actionPerformed(
      ActionEvent e) {
    try {
      result.setText(FareService.quote(
        plate.getText(), km.getText()));
    } catch (NumberFormatException ex) {
      result.setText("Enter a number for km.");
    } catch (VehicleNotFoundException ex) {
      result.setText(ex.getMessage());
    } catch (InvalidDistanceException ex) {
      result.setText(ex.getMessage());
    }
  }
}
```

Observed without a display: `No display: GUI not opened.`

One component has one parent. Add each field to the form once. Adding that same object to another panel moves it out of the first. Every BorderLayout region holds one component; a child JPanel lets that component contain a group [NUS].

Check yourself. Trace root CENTER → TerminalPanel NORTH → form → km. Trace a click to the unchanged FareService.quote call. The form grid keeps pairs aligned; the action flow keeps the button near its requested size.

Before your own theme: run the page 5 checks. On a school display, test normal → bad → corrected input; resize; use Tab and Space; quote a fare while Check ready is waiting. Then show the teacher your page 6 plan and use Thursday's final hour to build.

Sources [1 NTU] [2 HWS] [3 PRIN] [4 CMU] [5 NUS] (page 5). **Key validated facts:** Panels can contain panels [PRIN]; each panel has a manager, and position arguments must suit that manager [NUS].

LIPLET · THURSDAY After Step 4 on page 4, answer the THURSDAY strip (bottom of page 8) in pen. Detach only today's strip and submit it.

ERROR GUIDE | Keep this page beside your computer

Actual message excerpt	Meaning	Fix
error: cannot find symbol; symbol: class GridLayout	The layout class is not imported or is misspelled.	Add import java.awt.*; and check capital letters. Swing imports alone do not import AWT layout managers.
java.lang.IllegalArgumentException: rows and cols cannot both be zero	new GridLayout(0, 0) is invalid.	Use a positive row or column count, such as new GridLayout(0, 2).
NumberFormatException: For input string: "abc"	Distance text is not a number.	Keep the catch and show helpful feedback; correct the field and try again.
InvalidDistanceException: Use a finite km above 0.	Zero, negative, NaN or infinity is invalid.	Use a finite positive number. Keep the separate InvalidDistanceException catch.
VehicleNotFoundException: Unknown plate: ZZZ	No sample vehicle has that plate.	Use a known plate. Do not change the model merely to hide a typo.
java.awt.AWTError: Can't connect to X11 window server using ':0' as the value of the DISPLAY variable.	This workspace cannot reach its display server.	Use a school computer with a working display. A headless check cannot prove a window looks or responds correctly.

These messages were reproduced during preparation with JDK 21. Programs were compiled with Java 8 API and language compatibility. Line numbers, warnings and display-server wording can differ on school computers. The AWTError came from a separate forced-display probe, not a classroom program.

When the code compiles but the arrangement is wrong

Symptom	Check and repair
A control is missing	Use one direct child per BorderLayout region. Add related controls to a group panel first.
A label is beside the wrong field	Check GridLayout add order; pair label then field. Keep setLabelFor.
Flow controls vanish after narrowing	Rows wrap but still need enough height. Make the window taller; do not use setBounds or a null layout.
Text is clipped	Try 520 × 300, 720 × 320 and larger; inspect the longest normal feedback. Managers cannot fit unlimited text into a tiny window.
Button is silent or the window freezes	Retain registration and all catches. Keep sleep, join, await and slow file work out of listeners.

What was actually verified

Compilation: all complete examples passed Java 8 compatibility checks. **Program logic:** model rules, invalid input, recovery, readiness completion/interruption and fare actions during waiting passed. **Layout calculations:** Flow wrapping, equal cells and nested bounds passed; normal labels fitted at panel sizes 520 × 300, 720 × 320 and 900 × 480. **Window appearance:** not observed. **Resizing:** calculations checked; live resizing not observed. **User actions:** listener calls checked; physical clicks, focus order, keyboard navigation and closing not observed.

School checks: run java FareCheck; compare normal and invalid cases from page 3. In java Main test Tab, Shift+Tab and Space, bad → corrected input, resizing and Quote fare during readiness. Record actual results. Very long messages may still clip; use the longest normal feedback in your test.

STEP 5 Transfer the layout idea to your theme

Optional: add ThemeLayoutCheck.java; compile javac *.java; run java ThemeLayoutCheck. It checks grouped panels without a window or listener. Your own-theme task needs working model calls and actions. Vehicle still starts with java Main.

ThemeLayoutCheck.java — first half

```
import java.awt.*;
import javax.swing.*;
public class ThemeLayoutCheck {
  public static void main(String[] args) {
    SwingUtilities.invokeLater(new Runnable() {
      public void run() {
        JPanel form = new JPanel(
          new GridLayout(2, 2, 6, 6));
        form.add(new JLabel("Item:"));
        form.add(new JTextField("Rice", 8));
        form.add(new JLabel("Quantity:"));
        form.add(new JTextField("2", 8));
        JPanel actions = new JPanel(
          new FlowLayout());
        actions.add(new JButton("Calculate"));
```

ThemeLayoutCheck.java — continued

```
        JPanel root = new JPanel(
          new BorderLayout(6, 6));
        root.add(form, BorderLayout.CENTER);
        root.add(actions, BorderLayout.SOUTH);
        System.out.println("Form controls: "
          + form.getComponentCount());
        System.out.println("Action controls: "
          + actions.getComponentCount());
        System.out.println("Root groups: "
          + root.getComponentCount());
      }
    });
  }
}
```

Join both halves. Observed output: `Form controls: 4`; `Action controls: 1`; `Root groups: 2`. The example checks arrangement only, not a completed theme app.

Five opened university references used each teaching day

[1 NTU] Nanyang Technological University — GUI Programming. Excerpt: “layout manager”. Open source

[2 HWS] Hobart and William Smith Colleges — Basic Layout. Excerpt: “preferred size”. Open source

[3 PRIN] Princeton University — Graphical User Interfaces, pp. 7, 10–11. Excerpt: “Layout hierarchy”. Open source

[4 CMU] Carnegie Mellon University, hosted Java Tutorial — Using Layout Managers. Excerpt: “equal size”. Open source

[5 NUS] National University of Singapore, hosted Java Tutorial — How to Use Panels. Excerpt: “Adding Components”. Open source

CMU and NUS host older Java Tutorial material; cited layout concepts apply here. Use this module’s Java 8 EDT startup.

FRIDAY | Your Own Theme: Layout Performance Task and Presentation

Definition. An interface arrangement should serve its intended user. Extend your own theme program from Week 12 with useful groups of controls. Choose managers for a reason, preserve working rules and listeners, and prove the result with observed tests [HWS] [PRIN] [NUS].

> **ANALOGY** Arrange your own store, canteen, gadget, or school-service counter around the user's task. Show a classmate where to start without giving a spoken explanation.

Start Thursday after Step 4; present Friday. Extend your own theme; keep its useful model rules and existing workers. Learn all three managers, but use only those that serve your design.

Requirements checklist

☐ Extend your own theme and retain its model classes and useful actions.

☐ Use at least one JPanel inside another; name each group's purpose.

☐ Choose appropriate managers; explain your choices. You need not use all three.

☐ Keep clear labels, helpful feedback, and label-to-field associations.

☐ Preserve working listeners, correct model calls, and useful validation.

☐ Allow correction after invalid input without restarting.

☐ Create and update controls on the event dispatch thread (EDT).

☐ Keep existing slow work on a worker; retain safe updates and prevent duplicate jobs.

☐ Test normal, bad, corrected, resized, and keyboard-driven use; record actual results.

☐ Submit source, approved plan, test evidence and self-check; present your own code.

Plan before typing | Teacher checks first

Plan item	Your decision
Theme / intended user	_____
Existing model + method	_____
Groups + manager / reason	_____ _____
Input rules + feedback	_____
Worker behavior to retain	_____

Testing evidence | Write exact inputs

Test / expected	Actual / pass or fix
Normal input / model result	_____
Bad or blank / helpful error	_____
Corrected input / recovers	_____
Smaller/larger / readable	_____
Tab, Shift+Tab, Space / usable	_____
Other action during worker / responds, if used	_____

Self-check before presenting

Check	Yes / Not yet
Useful nested groups + managers	Yes / Not yet
Own model + actions + recovery	Yes / Not yet
EDT + safe worker, if used	Yes / Not yet
Readable, keyboard usable, evidence	Yes / Not yet

Two 3-hour days

Thursday: first 90 min: nesting and checks; next 30 min: Step 4 lab/slip; final 60 min: approved own-theme plan and first working arrangement.
Friday: first 90 min: retrieval, repair, peer tests, teacher check and rehearsal; final 90 min: presentations and Friday strip. For large classes use parallel circles of up to 15, with peer evidence and teacher rotation.

Also test one boundary and the longest normal message. Record the size used. Keep failures visible until fixed. If no worker is needed, explain why; retain any existing one safely.

Starter fragment | Fill blanks

```
JPanel form = new JPanel(new ___(2, 2, 8, 8));
JPanel actions = new JPanel(new ___());
JPanel root = new JPanel(new ___(8, 8));
root.add(form, BorderLayout.___);
root.add(actions, BorderLayout.___);
```

Fragment only, not a complete program. Add your labelled inputs to form, useful buttons to actions, and feedback to an appropriate region. Keep your existing listener and model call. Do not replace working rules with blanks.

Five-minute presentation

• Introduce your theme and intended user (30 s).
• Run one normal action (1 min).
• Show bad input, recovery, resizing, keyboard use; show responsiveness if applicable (2 min).
• Point to nested panels, managers and a retained model call (1 min).
• Answer one question (30 s).

Prepare to answer using your code

• Which group needs this manager, and why?
• Which panel is nested in which parent?
• How did you preserve listeners and model rules?
• What proves resizing and keyboard use work?
• Which test failed, what changed, and what remains?

Scoring rubric | 50 points

Criterion	Pts	Score
Useful arrangement and manager choices	10	___
Purposeful nested panels	8	___
Own model, actions, validation and recovery	10	___
EDT and preserved background behavior	8	___
Readability, keyboard use and test evidence	8	___
Presentation and own-code explanations	6	___
TOTAL	50	___

Sources [1 NTU] [2 HWS] [3 PRIN] [4 CMU] [5 NUS] (page 5). **Key validated facts:** Choose a layout to suit the task [HWS]; nested groups organize controls [PRIN] and the add method must match their manager [NUS].

> **LIPLET · FRIDAY** After Step 5 on page 5 and your own-theme presentation, answer the FRIDAY strip (top of page 8) in pen. Detach only today's strip and submit it.

LIPLET SHEET A · Print single-sided · Detach only today's strip

WEDNESDAY · NOV 18 Line Up the Form: GridLayout

Answer after Step 3 on page 3.
Name: _____ Section: _____ Score: _____ / 15

3.1 Define (5 pts): In Step 3, what does GridLayout(3, 2, 8, 8) mean? Describe the cell sizes.

3.2 Explain (5 pts): In Step 3, why must each label be added before its matching field? Give one limit of equal cells.

3.3 Connect (5 pts): In Step 3, which separate catch handles invalid distance? Explain why rearranging controls must preserve the Week 13 exception rules.

CUT ON TUESDAY · DETACH THE TUESDAY STRIP BELOW THIS LINE

TUESDAY · NOV 17 Give Each Group a Place: BorderLayout

Answer after Step 2 on page 2.
Name: _____ Section: _____ Score: _____ / 20

2.1 Define (5 pts): In Step 2, name the five BorderLayout regions and identify the one that receives remaining space.

2.2 Explain (5 pts): In Step 2, why are Check ready and its status placed in one JPanel before adding the group to SOUTH?

2.3 Identify (5 pts): Choose the region in Step 2: (a) heading; (b) TerminalPanel; (c) readiness group. (d) Name that group's manager.

2.4 Connect (5 pts): In Step 2, how do the retained ReadyTask and its invokeLater update connect to Week 16 threads and Week 17 responsiveness?

CUT ON MONDAY · DETACH THE MONDAY STRIP BELOW THIS LINE

MONDAY · NOV 16 Make Room: FlowLayout and Panels

Answer after Step 1 on page 1.
Name: _____ Section: _____ Score: _____ / 15

1.1 Define (5 pts): In Step 1, what is a layout manager? What does a JPanel group?

1.2 Explain (5 pts): In Step 1, what happens when a FlowLayout row runs out of width? Why might you also need more height?

1.3 Connect (5 pts): How does keeping the Week 17 Vehicle program while adding a separate layout demo connect to reusing classes from Weeks 11–12?

FRIDAY · NOV 20 Your Own Theme: Layout Performance Task and Presentation

Answer after Step 5 on page 5 and your presentation.
Name: _____ Section: _____ Score: _____ / 20

5.1 Point (5 pts): In your own theme code, identify a panel inside another panel. Name both managers and the line that connects your useful action to its listener.

5.2 Evaluate (5 pts): Defend one manager choice in your own code using your resizing or clipping test. Explain why another manager would serve that group less well.

5.3 Apply (5 pts): A school office needs two labelled inputs, an action row, and feedback. Adapt your own panel structure to this task. State what you would retain or change and why.

5.4 Reflect (5 pts): Compare your own program before and after this week. Describe one failed test, the repair, and evidence of the new result. Identify one remaining improvement.

CUT ON THURSDAY · DETACH THE THURSDAY STRIP BELOW THIS LINE

THURSDAY · NOV 19 Build a Clear Interface: Nested Panels

Answer after Step 4 on page 4.
Name: _____ Section: _____ Score: _____ / 15

4.1 Define (5 pts): In Step 4, what is a nested panel? Name the managers used for the form and action group.

4.2 Explain (5 pts): In Step 4, trace the km field through its parent panels. Why does one manager not control all descendants?

4.3 Connect (5 pts): Explain how Step 4 preserves the Week 12 model and the Week 16–17 worker. Which part changes, and which behavior stays?

Week 18 CodeShare practice additions

Use this appendix with the original module. Original lesson pages and sample listings remain unchanged. Text backups and data remain unchanged in the supporting-files folder.

Console checks exercise component state, layout or event behavior without opening a window. The message No display: GUI not opened. alone never completes an alteration. Check the actual interface on a school computer; browser GUI rendering and interaction remain unverified.

Run the unchanged sample first. Sample verified records the demonstration only. Make Your task changes, then compare the complete Expected task output. A successful alteration earns Completed. Keep module work, personal work and submitted versions separate. Paper liplets remain teacher assessed.

Step 1 alteration

Sample: java-2-w18-d1-s1 Task: java-2-w18-d1-s1-alter-v1

Build a second arrangement using GridLayout with two rows and two columns. Add the same terminal label and three vehicle buttons. Print its actual layout dimensions and control count. Keep the original FlowLayout window demo for comparison.

Before the existing headless check in `LayoutDemo.java`, add this console check. Keep the original sample message and the window code.

```
JPanel grid = new JPanel(new GridLayout(2, 2, 8, 8));
grid.add(new JLabel("Terminal:"));
grid.add(new JButton("Jeepney"));
grid.add(new JButton("Tricycle"));
grid.add(new JButton("Bus"));
GridLayout layout = (GridLayout) grid.getLayout();
System.out.println("Grid: " + layout.getRows() + " x " + layout.getColumns());
System.out.println("Controls: " + grid.getComponentCount());
```

Expected task output

```
Grid: 2 x 2
Controls: 4
No display: GUI not opened.
```

Step 2 alteration

Sample: java-2-w18-d2-s2 Task: java-2-w18-d2-s2-alter-v1

Move the readiness group from SOUTH to EAST in createContent(). Print how many controls occupy the EAST group.

Before the existing headless check in `Main.java`, add this console check. Keep the original sample message and the window code.

```
JPanel root = createContent();
JPanel east = (JPanel) ((BorderLayout) root.getLayout()).getLayoutComponent(BorderLayout.EAST);
System.out.println("East controls: " + east.getComponentCount());
```

Expected task output

```
East controls: 2
No display: GUI not opened.
```

Step 3 alteration

Sample: java-2-w18-d3-s3 Task: java-2-w18-d3-s3-alter-v1

Change TerminalPanel from three rows and two columns to two rows and three columns. Print its actual GridLayout dimensions.

Before the existing headless check in `Main.java`, add this console check. Keep the original sample message and the window code.

```
java.awt.GridLayout grid = (java.awt.GridLayout) new TerminalPanel().getLayout();
System.out.println("Grid: " + grid.getRows() + " x " + grid.getColumns());
```

Expected task output

```
Grid: 2 x 3
No display: GUI not opened.
```

Step 4 alteration

Sample: java-2-w18-d4-s4 Task: java-2-w18-d4-s4-alter-v1

Move the Quote fare action group from SOUTH to EAST within TerminalPanel. Print the actual number of EAST controls.

Before the existing headless check in `Main.java`, add this console check. Keep the original sample message and the window code.

```
TerminalPanel panel = new TerminalPanel();
javax.swing.JPanel east = (javax.swing.JPanel) ((java.awt.BorderLayout)
panel.getLayout()).getLayoutComponent(java.awt.BorderLayout.EAST);
System.out.println("East controls: " + east.getComponentCount());
```

Expected task output

```
East controls: 1
No display: GUI not opened.
```

Step 5 alteration

Sample: java-2-w18-d5-s5 Task: java-2-w18-d5-s5-alter-v1

Add a Reset button to the existing actions panel. Keep the form and root groups unchanged; print their actual counts as before.

Expected task output

```
Form controls: 4
Action controls: 2
Root groups: 2
```